Lecture 7: Synthetic Data and Modern Post-Training Methods

RLHFBOOK.COM

Nathan Lambert

Post-training course. Chapter 12 on Synthetic Data.

Why synthetic data took over

When the first models were trained with RLHF, human data was *the only* way to get high-quality responses and reliable feedback. As models got better, that assumption broke down fast.

- **Cheaper, faster iteration** – synthetic data lowered the price of a post-training experiment, opening the field to everyone who was priced out of human-data pipelines. The time-to-collect it is also far faster, enabling the RSI arguments we hear today.
- **A capability threshold** – synthetic data in post-training only worked once GPT-4-class models arrived. Llama 2 and GPT-3.5-Turbo were not reliable enough to generate *or* supervise data; the LLM-as-a-judge ability emerged in the GPT-3.5 → GPT-4 jump.
- **The center of gravity of post-training** – today, leading models *need* synthetic data to reach the frontier. Distillation is the general word for how to transfer capabilities from a stronger model to a student.

This lecture

We survey how synthetic data has replaced or expanded much of the post-training pipeline – then derive **on-policy distillation** from scratch as the technical core.

Definition: Synthetic data is any training data used for an AI model that is generated in some part from another AI model. This is very broad!

The plan, roughly

1. The **roles** of synthetic data
2. **General distillation** with synthetic data
3. The path to **on-policy distillation** (the technical core)
4. **AI feedback** & **Constitutional AI**
5. **Rubrics** – prompt-specific AI feedback

Lecture 7: Where it sits

Overview

1. Introduction
2. Key related works
3. Training overview

Core training pipeline

4. Instruction tuning
5. Reward models
6. Reinforcement learning
7. Reasoning
8. Direct alignment
9. Rejection sampling

Data & preferences

10. What are preferences
11. Preference data
12. **Synthetic data & CAI**

Practical considerations

13. Tool use
14. Over-optimization
15. Regularization
16. Evaluation
17. Product & character

Appendices

- A. Definitions
- B. Style & information
- C. Practical issues

Course home

- rlhfbook.com
- GitHub repo

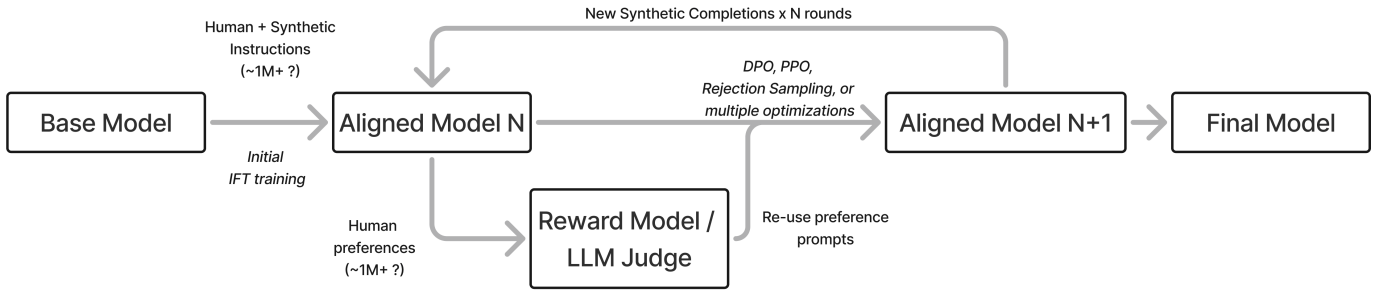
Recall: Where synthetic data sits in a pipeline

The post-training pipeline is many moving parts:

1. Collect / generate **prompts**
2. Generate **completions** for SFT
3. Collect **preferences** for RLHF
4. Score **rewards** for RLVR
5. Filtering, cleaning, all of the above

Synthetic data from other language models now feed every box: writing prompts from seeds, generating completions, labeling preferences, and verifying answers for RL.

We will cover a few training methods that emerged around these ideas too.



Modern post-training runs many rounds of feedback and optimization; synthetic data now feeds most of the loop.

Part 1: The roles of synthetic data

What synthetic data is used for

"Synthetic data" in modern post-training spans the whole pipeline – a single model is reused for many roles:

- **Generate new prompts** from seed examples (Wang et al., 2023)
- **Modify / expand** existing prompts
- **Generate completions** to prompts (Beeching et al., 2024)
- **Provide AI feedback** to create preference data (Cui et al., 2023)
- **Filter completions** for quality (Li et al., 2024)
- **Verify** answers as rewards for RL

The balance between human and synthetic data

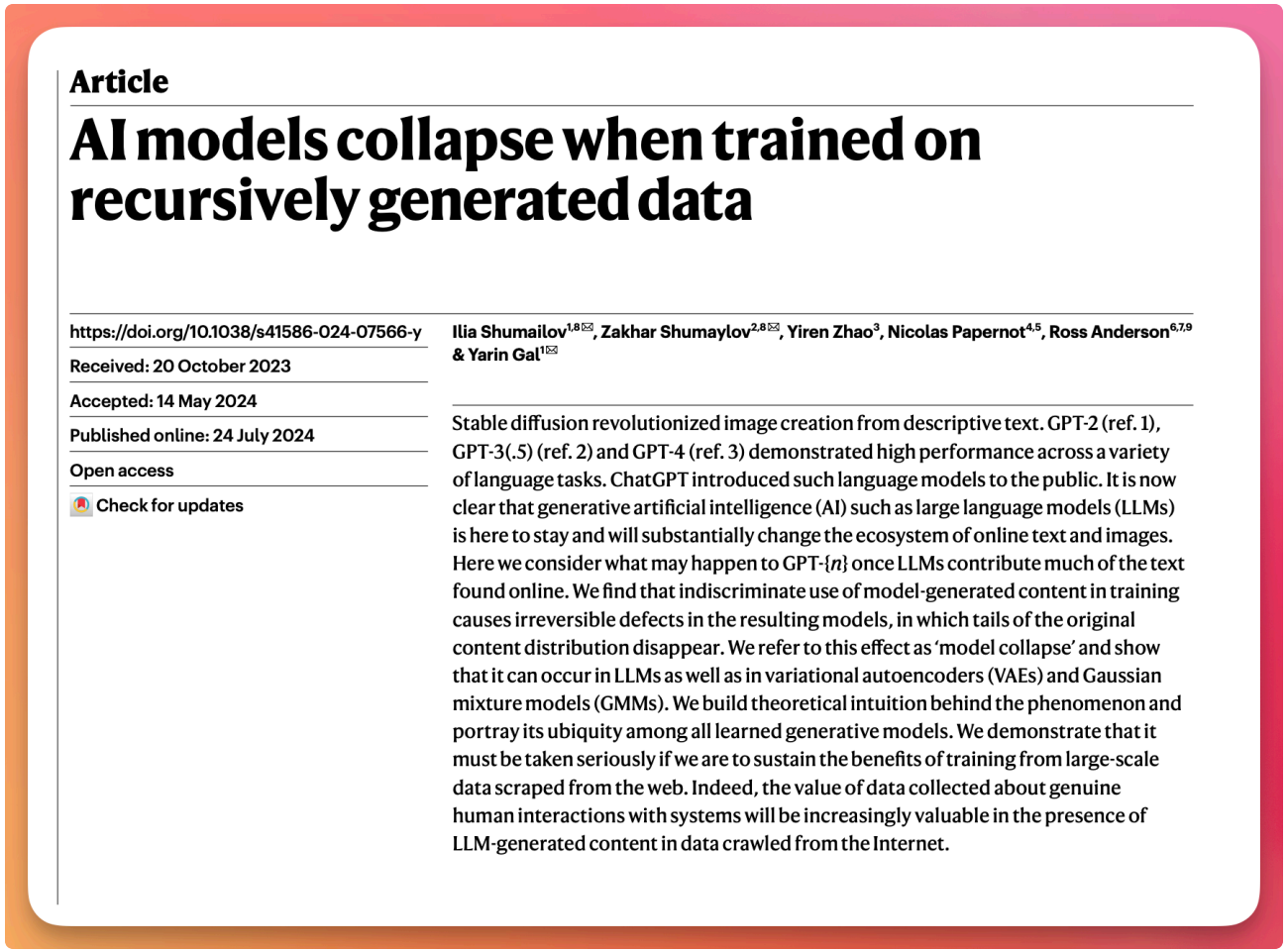
Synthetic data has not replaced human data uniformly across the pipeline.

- **Instruction data (SFT):** synthetic has largely *won* – distillation beats most human writers at scale.
- **Preference data (RLHF):** *mixed* – academic work shows it performs comparably, yet frontier labs treat human preference data as a competitive moat.
- **Evaluation:** LLM-as-a-judge scales *scoring* cheaply, but benchmarks and ground-truth labels still need human grounding/correlation.

Human data curation is heavily involved in determining the frontier of models and seeding initial progress, then synthetic data is used to scale it.

Around the launch of ChatGPT, human data was a central driver of progress.

Model collapse: An outdated worry



Article

AI models collapse when trained on recursively generated data

https://doi.org/10.1038/s41586-024-07566-y

Received: 20 October 2023

Accepted: 14 May 2024

Published online: 24 July 2024

Open access

Check for updates

Ilia Shumailov[1,8]✉, Zakhar Shumaylov[2,8]✉, Yiren Zhao[3], Nicolas Papernot[4,5], Ross Anderson[6,7,9] & Yarin Gal[1]✉

Stable diffusion revolutionized image creation from descriptive text. GPT-2 (ref. 1), GPT-3(.5) (ref. 2) and GPT-4 (ref. 3) demonstrated high performance across a variety of language tasks. ChatGPT introduced such language models to the public. It is now clear that generative artificial intelligence (AI) such as large language models (LLMs) is here to stay and will substantially change the ecosystem of online text and images. Here we consider what may happen to GPT-{*n*} once LLMs contribute much of the text found online. We find that indiscriminate use of model-generated content in training causes irreversible defects in the resulting models, in which tails of the original content distribution disappear. We refer to this effect as 'model collapse' and show that it can occur in LLMs as well as in variational autoencoders (VAEs) and Gaussian mixture models (GMMs). We build theoretical intuition behind the phenomenon and portray its ubiquity among all learned generative models. We demonstrate that it must be taken seriously if we are to sustain the benefits of training from large-scale data scraped from the web. Indeed, the value of data collected about genuine human interactions with systems will be increasingly valuable in the presence of LLM-generated content in data crawled from the Internet.

Recursive self-training narrows the distribution over generations; the tails go first. Source: Shumailov et al. (2024).

A common criticism: repeatedly training on a model's own generations can narrow the effective training distribution (Shumailov et al., 2024).

The argument follows as: *As diversity drops, rare facts and styles are underrepresented and small mistakes compound across iterations.*

Model collapse: An outdated worry

But collapse is mostly a failure of *unfiltered, single-model, self-training* loops. In practice it is avoided by:

- mixing in real / human data,
- using diverse teachers,
- deduplication,
- strong quality filters.

Evidence suggests synthetic data can – and should – be used at scale without the catastrophic regressions of the strongest collapse story (Gerstgrasser et al., 2024) (Feng et al., 2024).

Canonical, early synthetic datasets and their scale

A few datasets defined each era: **UltraFeedback** (Cui et al., 2023) (kickstarted the DPO revolution), **Stanford Alpaca** (early chat SFT), **Tülu 3** (Lambert et al., 2024) (pre RLVR SFT set), and **OpenThoughts 3** (Guha et al., 2025) (general reasoning set).

	Prompts	Tokens (approx.)
Stanford Alpaca (2023)	52K	~10M
Tülu 3 (2024)	~1M	~500M
OpenThoughts 3 (2025)	1.2M	~10B

Two meanings of “distillation”

Technical (Knowledge Distillation): train a smaller *student* to match a stronger *teacher’s* full output distribution – *soft* labels, not one-hot targets (Hinton et al., 2015). More on this later in the lecture.

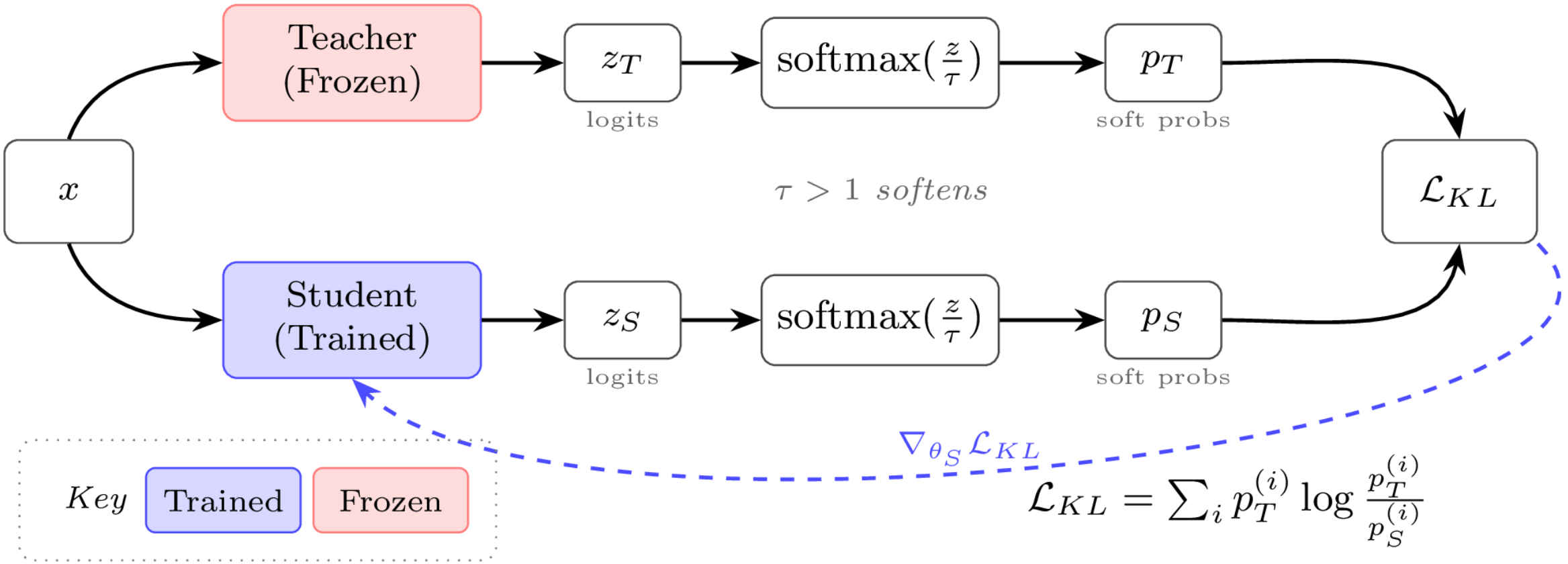
Colloquial (today’s usage): “train a weaker model on the outputs of a stronger model.” It’s what people mean when they say something like “Chinese models distill from GPT/Claude.”

Aside: Distillation on Interconnects this year

Further reading

- **How much does distillation really matter for Chinese LLMs?** (Feb 2026) - how much of Chinese open models' gains actually trace to distilling frontier APIs.
- **The distillation panic** (May 2026) - why reacting quickly to frontier lab's fear-based messaging on distillation could be bad for the broader AI ecosystem.

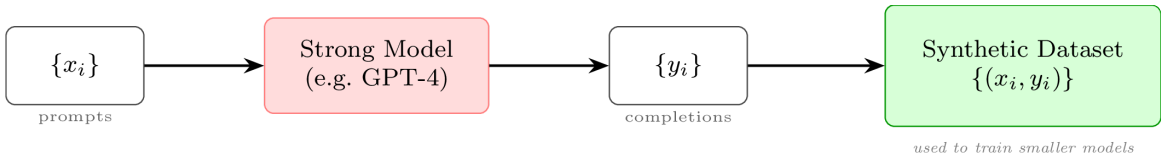
Distillation 1: Classic knowledge distillation



Knowledge distillation trains a student to match the soft probability distribution of a larger teacher via KL divergence. Both models see the same input, and temperature scaling () softens the distributions to expose relationships between classes.

Hinton et al., 2015

Distillation 2: The synthetic-data generation pipeline



Prompts are passed through a strong model to generate completions, which are paired into a training dataset and used to fine-tune smaller models with standard supervised learning. More complex pipelines edit completions, generate preference pairs, or filter for quality.

Yes, this is intentionally very simple as a diagram.

What changed with reasoning models

Early on this was simple: train on a strong model's API outputs.

Reasoning models made it harder – you want the *reasoning traces*, and closed APIs stopped returning them. Distillation shifted to:

- **More aggressive jailbreaking** to recover hidden traces (controversial).
- **Open models** more often – having the weights lets you get the full trace.

Part 2: The path to on-policy distillation (OPD)

Setup and notation

Knowledge distillation, introduced by Hinton, Vinyals & Dean (Hinton et al., 2015), in its original form uses **soft** labels – the full distribution over next tokens – rather than the one-hot target of next-token prediction. To apply it to autoregressive LMs, decompose the loss per token.

Let:

- s – the source prompt
- $u = (u_1, \ldots, u_J)$ – a complete output sequence from the **teacher**
- $\mathcal{V}$ – the output vocabulary (tokenizer)
- q – the teacher's next-token distribution, p – the student's

Notation discipline: u is a *teacher* trajectory; we reserve a for the *student*-sampled completion in the on-policy / RL notation later. This u-vs-a split is the whole point of "on-policy."

Word-level (per-token) distillation

Standard teacher-student distillation for an LLM (Kim & Rush, 2016) – the classic Hinton soft-label idea applied at every token position:

$$\mathcal{L}_{\text{WORD-KD}} = -\sum_{j=1}^{J}\sum_{k=1}^{|\mathcal{V}|} q(u_j = k \mid s, u_{<j}) \log p(u_j = k \mid s, u_{<j}).$$

At each position, this matches the teacher's *full* next-token distribution over the entire vocabulary on a pre-existing training corpus $\mathcal{V}$ – soft labels, not the one-hot target. The inner $\sum_{k=1}^{|\mathcal{V}|}$ runs over *every possible next token*.

Matching a distribution over every token sounds expensive, but it is **tractable**: it is just $|\mathcal{V}|$ probabilities per position – the same $O(J\,|\mathcal{V}|)$ cost as ordinary cross-entropy. Matching over whole *sequences* is the hard part (next slide).

Read "word-level" as per-token over the tokenizer.

Sequence-level distillation

Word-level KD gives soft per-token distributions. The goal of sequence-level distillation from the paper is to apply this to data that the teacher generated, providing fresh training data/signal, and improving performance!

(WORD-KD is the baseline in the Kim & Rush paper.)

Sequence-level distillation

Sequence-level KD instead approximates the teacher's distribution over *whole sequences* $\mathcal{U}$ – an intractable sum over exponentially many sequences – by its mode: the *teacher* generates one high-probability output $\hat{u} = \mathrm{BeamSearch}_q(s)$ and the student trains on it with plain NLL:

$$\begin{aligned}
\mathcal{L}_{\text{SEQ-KD}}(s) &= -\sum_{u \in \mathcal{U}} q(u \mid s) \log p(u \mid s) && \text{sum over all sequences: intractable} \\
&\approx -\log p(\hat{u} \mid s) && \text{teacher} \approx \text{point mass on } \hat{u} \\
&= -\sum_{j=1}^{|\hat{u}|} \log p(\hat{u}_j \mid s, \hat{u}_{<j}) && \text{autoregressive factorization}
\end{aligned}$$

Aside: A first wave of offline-KD models were classifiers – DistilBERT (Sanh et al., 2019) and TinyBERT (Jiao et al., 2020) – pairing offline distillation with other LM advances. Not *sequence* distillation because these are embedding models, but building on related momentum in the area.

Finding a connection between SEQ-KD and SFT

To start, recall the cross-entropy of a teacher q and student p – the same form as the KD losses:

$$H(q, p) = -\sum_{z} q(z) \log p(z) \quad \text{definition}$$

Add and subtract the teacher's own log-probabilities, $\sum_z q(z) \log q(z)$ – does not involve p at all:

$$H(q, p) = \underbrace{-\sum_{z} q(z) \log q(z)}_{H(q)} + \sum_{z} q(z) \log q(z) - \sum_{z} q(z) \log p(z) \quad \text{add and subtract}$$

The last two sums share the weight $q(z)$, so they fold into one log-ratio (log rules):

$$H(q, p) = H(q) + \sum_{z} q(z) \log \frac{q(z)}{p(z)} = H(q) + D_{\mathrm{KL}}(q \| p) \quad \text{definition of KL}$$

Finding a connection between SEQ-KD and SFT

We just showed cross-entropy decomposes into the teacher's entropy plus a KL:

$$H(q, p) = H(q) + D_{\mathrm{KL}}(q \| p)$$

$H(q)$ depends only on the fixed teacher, so minimizing cross-entropy *is* minimizing the forward KL:

$$\boxed{\min_p H(q, p) \;\equiv\; \min_p D_{\mathrm{KL}}(q \| p) \quad (\text{forward KL: the direction of offline KD and SFT})}$$

So sequence-level KD reduces to SFT on the teacher's generated text – "offline KD," generations produced often ahead of time by a teacher model.

Exposure bias in offline KD: The train / test mismatch

Offline KD samples **teacher** trajectories $u \sim \pi_T$ and matches per-token (here $q = \pi_T, p = \pi_\theta$):

$$\mathcal{L}_{\mathrm{KD}}(\theta) = \mathbb{E}_{s\sim\mathcal{D},\, u\sim\pi_T(\cdot|s)} \sum_t D_{\mathrm{KL}}\big(\pi_T(\cdot \mid s, u_{<t}) \,\|\, \pi_\theta(\cdot \mid s, u_{<t})\big)\,.$$

But at test time the student rolls out under its own policy ($\ell_{\mathrm{task}}(s,a)$ is the loss function for that test-time domain):

$$\mathcal{L}_{\mathrm{eval}}(\theta) = \mathbb{E}_{s\sim\mathcal{D}_{\mathrm{test}},\, a\sim\pi_\theta(\cdot|s)}\ \ell_{\mathrm{task}}(s,a).$$

Since $\pi_T \neq \pi_\theta$, training and test prefixes come from different state distributions – **exposure bias** is the propensity for the student to accumulate errors (Arora et al., 2022) (Song & Zheng, 2026).

The DAgger analogy: Compounding error

On-policy distillation connects to **imitation learning**: DAgger trains an agent on its own rollouts, with an oracle (teacher) labeling the action it *should* have taken (Ross et al., 2011).

Suppose the student matches the teacher within per-step error ϵ on teacher-induced states:

$$\mathbb{E}_{s_t \sim d_{\pi_T}} \left[\mathbb{I}(\pi_\theta(s_t) \neq \pi_T(s_t)) \right] \leq \epsilon.$$

The supervised imitation-learning analysis (Ross et al., 2011) shows that the expected loss accumulated along a length-L trajectory sampled from the student can scale quadratically in L (Song & Zheng, 2026):

$$\mathbb{E}_{a \sim \pi_\theta(\cdot|s)} \left[\sum_{t=1}^{L} \ell(s, a_{<t}) \right] \leq O(\epsilon L^2).$$

For LLMs this is more of an analogy – token losses are distributional (KL), not 0-1 action disagreement (indicator function in the first equation on this slide returns 0 or 1).

From off-policy to on-policy

Sampling from the student rather than the teacher minimizes a lot of the distributional errors we've covered.

- In offline KD, a single suboptimal token can nudge the student generation slightly **out-of-distribution**; the model, never having seen that token in training, is more likely to err again.
- On-policy distillation iteratively samples from the student and supervises it with the teacher at *its own* visited states.
- Under DAgger's interactive analysis, this drops compounding from $O(\epsilon L^2)$ to $O(\epsilon L)$ (Ross et al., 2011).
- **MiniLLM** introduced a reverse-KL objective inside a policy-gradient frame (Gu et al., 2024); concurrent work connected on-policy KD to imitation learning (Agarwal et al., 2024) (this paper is closer to the modern distillation form used today).

The on-policy distillation objective

Let $a = (a_1, \ldots, a_L)$ be a completion sampled from the **student** $\pi_\theta(\cdot \mid s)$, with token-level state $s_t = (s, a_{<t})$. The teacher π_T is fixed:

$$\mathcal{L}_{\text{OPD}}(\theta) = \mathbb{E}_{s,\, a \sim \pi_\theta(\cdot|s)} \sum_t D_{\text{KL}}(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_T(\cdot \mid s_t))$$

This is now in the sampling / expectation framework of Chapter 6 (policy gradients) – a natural bridge to modern RL training infrastructure that alternates generate-and-update.

Sampling from the *student* is also what flips the KL direction we minimize (forward $D_{\text{KL}}(\pi_T \,\|\, \pi_\theta)$ → reverse $D_{\text{KL}}(\pi_\theta \,\|\, \pi_T)$ – next slide).

Forward vs. reverse KL

Sampling completions from the student is what puts π_θ on the **left** of the KL – which flips its direction (estimating the KL and its direction relies on which distribution you sample from):

Offline KD / SFT (forward KL) – the expectation is over the *teacher*, $z \sim \pi_T$ (**off-policy**: a fixed teacher dataset):

$$D_{\mathrm{KL}}(\pi_T \,\|\, \pi_\theta) = \mathbb{E}_{z\sim\pi_T}\left[\log \frac{\pi_T(z)}{\pi_\theta(z)}\right]$$

Mass-covering – weighted by **teacher** mass: wherever the teacher has mass and $\pi_\theta \to 0$, the log-ratio blows up, so the student must cover *everything* the teacher might say.

On-policy distillation (reverse KL) – the expectation is over the *student*, $z \sim \pi_\theta$ (**on-policy**: you sample the model you're training):

$$D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_T) = \mathbb{E}_{z\sim\pi_\theta}\left[\log \frac{\pi_\theta(z)}{\pi_T(z)}\right]$$

Mode-seeking – weighted by the **student's** own mass: penalized only where *it* puts probability the teacher dislikes, so it collapses onto the teacher's modes. (Why reverse KL is often better: Chapter 15.)

KD as an RL advantage

Recent implementations take the KD distance directly as a reward: substitute the negative per-token reverse-KL contribution as the advantage (Lu & Thinking Machines Lab, 2025). The reverse KL at state s_t is an expectation over *student*-sampled tokens:

$$D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_T(\cdot \mid s_t)) = \mathbb{E}_{a_t \sim \pi_\theta(\cdot|s_t)}\left[\log \pi_\theta(a_t \mid s_t) - \log \pi_T(a_t \mid s_t)\right].$$

In practice you never sum over the vocabulary: the single sampled token is an unbiased estimate $\hat{D}_{\mathrm{KL}}$ of that KL (Schulman, 2020), and the per-token advantage is its negative:

$$A_t^{\mathrm{OPD}} = -\hat{D}_{\mathrm{KL}} = -\left(\log \pi_\theta(a_t \mid s_t) - \log \pi_T(a_t \mid s_t)\right) = \log \pi_T(a_t \mid s_t) - \log \pi_\theta(a_t \mid s_t).$$

KD as an RL advantage

In practice you never sum over the vocabulary: the single sampled token is an unbiased estimate $\hat{D}_{\mathrm{KL}}$ of that KL (Schulman, 2020), and the per-token advantage is its negative:

$$A_t^{\mathrm{OPD}} = \log \pi_T(a_t \mid s_t) - \log \pi_\theta(a_t \mid s_t).$$

- Tokens more likely for the teacher → positive advantage; less likely → negative.
- Same form, opposite KL: $\log \pi_T - \log \pi_\theta$ *looks* like a forward-KL term – what makes it **reverse** KL is sampling $a_t \sim \pi_\theta$ (the student), not the sign.
- The teacher log-prob gap is dense, token-level feedback – potentially richer than a sparse verifiable reward or a single scalar reward-model score.
- This layers into modern RL machinery – e.g. add it alongside GRPO's group-level normalization for more complex reward shaping.

Multi-teacher on-policy distillation (MOPD)

Use several teachers – domain specialists (math, code) or earlier checkpoints – each with a per-prompt mixture weight $w_k(s)$ (with $\sum_k w_k(s) = 1$) (LLM-Core Xiaomi, 2025):

$$\mathcal{L}_{\text{MOPD}}(\theta) = \mathbb{E}_{s,\, a\sim\pi_\theta(\cdot|s)} \sum_t \sum_k w_k(s)\, D_{\text{KL}}\big(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_{T_k}(\cdot \mid s_t)\big)\,.$$

At scale, this lets a growing org divide labor: many groups train expert teachers that later distill into one final student (DeepSeek-V4-Pro (DeepSeek-AI, 2026), MiMo-V2-Flash (LLM-Core Xiaomi, 2025)).

For more, see the conversation I had with Finbarr Timbers that maps MOPD across the 2026 frontier recipes.

Self-distillation: Pushing the frontier

At the absolute frontier there is no stronger model to distill from. **On-Policy Self-Distillation (OPSD)** sidesteps this: the teacher is the *same model conditioned on privileged information* – a hint the student model won't have at inference (Zhao et al., 2026). The self-distillation gradients will teach the model that tokens after the hint were a mistake, absorbing the lesson with an OPD-style loss.

Cursor's Composer 2.5 (from Kimi K2.5) trained this way (Team, 2026) (highly recommend watching this video!):

- A judge reviews RL trajectories against a list of common bugs.
- On a bug, it inserts a hint into the sequence – privileged information the model wouldn't see at test time.
- The model takes a KD loss toward its own hinted continuation, learning to reach it *unaided*.
- A hint in token space is enough to self-correct – *how* to structure that signal is an active area ("privileged information") (Penaloza et al., 2026).

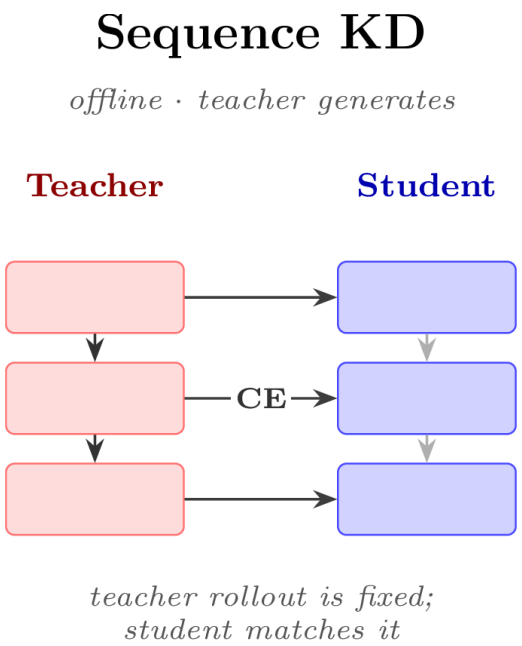
On-policy distillation is becoming very popular

A resurgence of teacher-student KD has accompanied the shift toward reasoning and agentic models. Leading models trained with new forms of knowledge distillation include:

- **Qwen3** (Alibaba) (Yang et al., 2025)
- **MiMo-V2-Flash** (Xiaomi) (LLM-Core Xiaomi, 2025) (introduced MOPD)
- **DeepSeek-V4-Pro** (DeepSeek-AI, 2026)

One caveat: per-token KD needs the student and teacher to share a tokenizer – unusual among post-training methods, and part of why it thrives inside a lab's own model family.

From offline KD to self-distillation

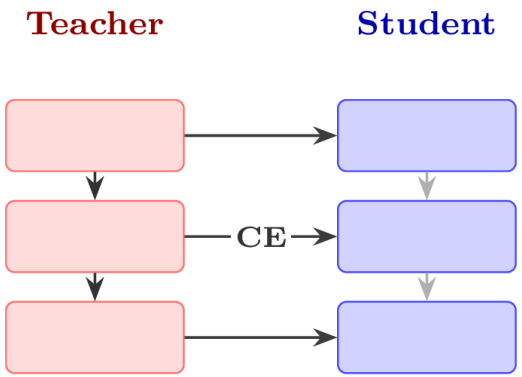


Sequence KD: the teacher generates the rollout offline, and the student is trained to match those fixed targets with a cross-entropy loss.

From offline KD to self-distillation

Sequence KD

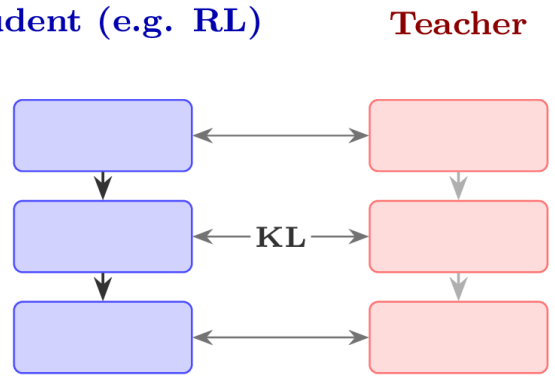
offline · teacher generates



teacher rollout is fixed; student matches it

On-policy distillation

on-policy · student generates (e.g. RL)



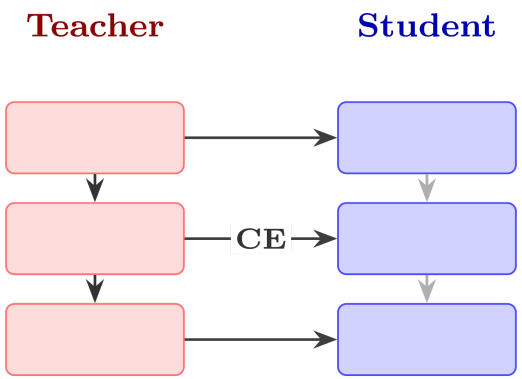
student rollout; teacher scores each token: $D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_T)$

On-policy distillation flips who generates: the rollout comes from the student (e.g. inside an RL loop), and a separate teacher scores every visited token with a per-token KL.

From offline KD to self-distillation

Sequence KD

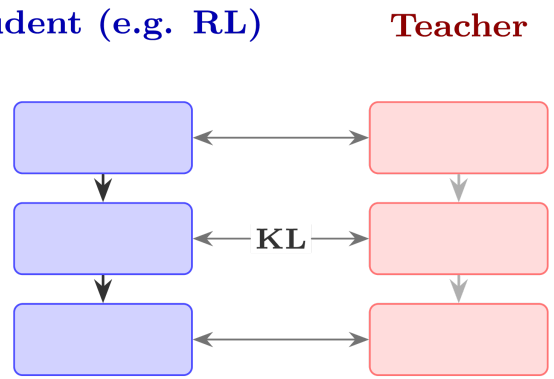
offline · teacher generates



teacher rollout is fixed; student matches it

On-policy distillation

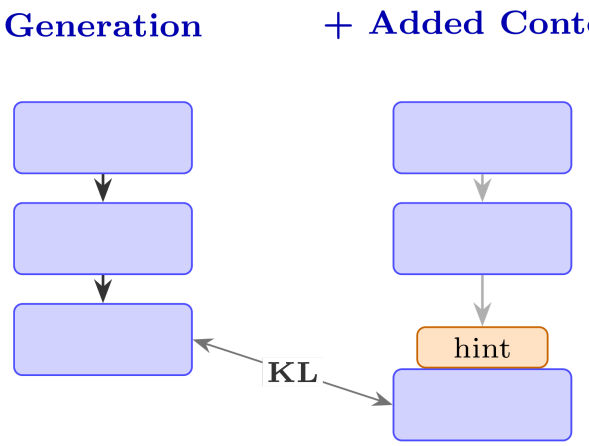
on-policy · student generates (e.g. RL)



student rollout; teacher scores each token: $D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_T)$

On-policy self-distillation

Privileged information creates the teacher trajectory



On-policy distillation loss with no separate teacher

On-policy self-distillation: one model plays both roles – privileged information (a hint) added to the context creates the teacher trajectory, with no separate teacher model.

Part 3: AI feedback & Constitutional AI

Reinforcement learning from AI feedback (RLAIF)

Soon after RLHF took off, **RL from AI Feedback (RLAIF)** emerged – using AIs to approximate the human-data step, starting with pairwise preferences (Lee et al., 2023) (Sharma et al., 2024) (Castricato et al., 2024).

After initial debates if this worked well, eventually it became the default. Cost was one of the obvious advantages (estimates):

- One piece of *human* preference data: **$1 – $10+** per prompt.
- *AI* feedback (e.g. GPT-4o): **< $0.01** per prompt.

Human labor cost is roughly flat; model price-per-performance keeps dropping. This opened RLHF experimentation to a population previously priced out.

The bias-variance tradeoff

I've heard a colloquial rule of thumb in early RLHF v RLAIF debates. There's an intuitive nature to it.

Human data – *high-noise, low-bias.*

Harder to collect and filter, but when wrangled it gives a very reliable signal.

Synthetic preference data – *low-noise, high-bias.*

Easier to start with, but can carry tricky second-order effects that are *systematically* baked into the data.

Balancing human and AI feedback

No clear literature on the ultimate limits between human and AI preference data. Some context includes:

- Early RLAIF literature claimed AI feedback could fully replace human data – especially on chat tasks (Lee et al., 2023) (Cui et al., 2023).
- Later work is more nuanced: on broader evaluations (incl. reasoning), the best mix routes hard data points to humans while sending most to AI (Miranda et al., 2025) (Xu et al., 2025).
- No study has mapped the human/AI balance across *all* domains, highlighting a general limitation of open academic work to wrangle high-quality human data.
- **Industry reality:** human preference data is still treated as a substantial moat. Could be more from prompt distributions, implicit feedback, etc. Human data still is used.

Building specialized judge models

If you're using substantial AI feedback or LLM-as-a-judge evaluations, the question arises as to if you should have a specialized model for that purpose. The question is – how well do they work?

- Some research is done to understand the performance of LLMs in these feedback domains. Results include how LLMs are inconsistent evaluators (Wang et al., 2024) and show **self-preference bias** – they favor their own generations (Panickssery et al., 2024).
- Dedicated judge / critic models exist – Prometheus (Kim et al., 2024), Prometheus 2 (Kim et al., 2024), and others – but are not widely adopted in documented post-training recipes.
- Equilibrium: Frontier models are already trained hard for judging, so you rarely need your own – *unless* your task has private data not on the public internet.

CAI: The earliest large-scale synthetic RLHF data

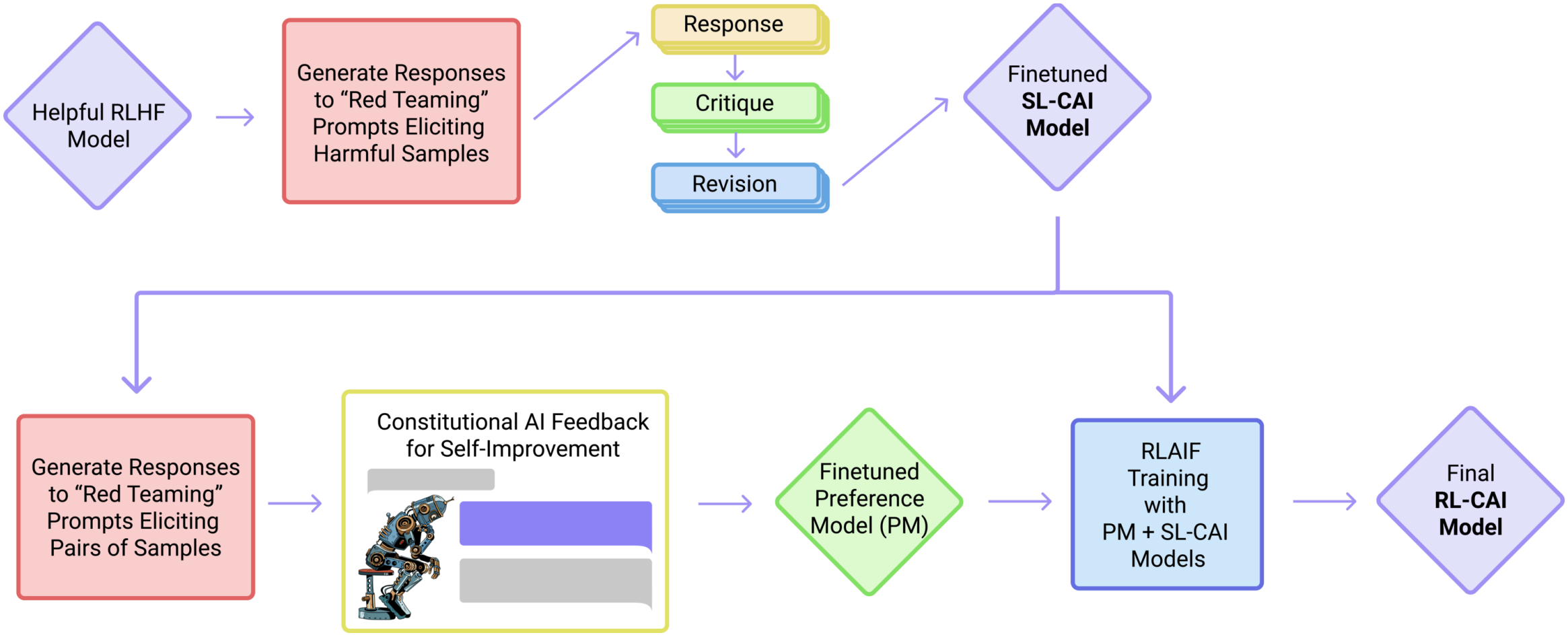
Constitutional AI (CAI) – Anthropic's post-training method for the Claude models – is the earliest documented, large-scale use of synthetic data for RLHF (Bai et al., 2022). CAI refers to a specific set of techniques for their early Claude models, and definitely has shifted substantially (though Anthropic still uses a constitution – yes, confusing).

The term **RLAIF** was coined in this paper as well, prompting confusion on the relation of the two.

> *CAI is the example that kickstarted the broader field of RLAIF. CAI $\subset$ RLAIF.*

CAI generates synthetic data in two ways – one for instructions, one for preferences. The well-known and more influential part of it is preferences.

Constitutional AI: The original diagram



Two stages. Supervised (top): the model critiques and revises its own red-teamed responses against a constitution, fine-tuning into SL-CAI. RL (bottom, RLAIF): a preference model trained on AI comparisons drives RL into the final RL-CAI model. Source: Bai et al. (2022).

Bai et al., 2022

Stage 1: Critique and revise → SFT data

A **constitution** $\mathcal{C}$ is a human-written set of principles (e.g. *"Is the answer encouraging violence?"*, *"Is the answer truthful?"*).

The model repeatedly samples a principle $c_i \in \mathcal{C}$ and revises its latest output y^i to the prompt x to align with c_i:

$$\{c_0, c_1, \ldots, c_{n-1}\} \longrightarrow \{y^0, y^1, \ldots, y^n\} \qquad \Longrightarrow \qquad \text{SFT point } (x, y^n).$$

The model is then fine-tuned on the refined dataset. These critique methods are also used broadly for data filtering and synthetic-data generation.

Stage 2: AI preference labels → RLAIF

Construct preferences by giving a feedback model:

- a prompt x,
- a subset of principles $\{c_0, \ldots, c_n\}$,
- two completions y_0, y_1 labeled (A) / (B).

The model selects which answer is higher quality and more aligned with the principle. Then RLHF proceeds as normal – hence *RLAIF*.

Aside: Also linked to literature like generative reward models and progression in the LLM-as-a-judge field. See Chapter 5 / Lecture 2.

- **Earlier:** prompt with `The answer is:` and read which of A / B has higher token probability.
- **Modern:** a **generative reward model** explains its reasoning, then selects (Mahan et al., 2024) (cf. principle-guided reward models (Sun et al., 2024)).

Part 4: Rubrics – prompt-specific AI feedback

Why did rubrics become popular?

Rubrics became a popular tool for scaling RL on the long-tail of domains. They're also used to help with domain-specific evaluations and any other place domain expertise needs to be "trained into" the models.

- A way to extend ideas from RL with verifiable rewards (Chapter 7) to tasks *without* clearly verifiable answers.
- Write nearly-verifiable criteria for a prompt, generate multiple answers, and RL-update toward the best ones.
- Emerged in late 2024 → 2025 as LLM judges and synthetic-data practices matured. Also likely a function of making RL more broadly accessible to frontier post-training.
- Already delivering gains in scientific reasoning and factuality (Gunjal et al., 2025) (Viswanathan et al., 2025) (Rezaei et al., 2025) (Liu et al., 2025).

A rubric example

For a prompt with no single right answer, score against tagged criteria (Liu et al., 2025). An example, abbreviated rubric is below:

```
Prompt: As a museum curator, suggest five obscure artifacts for a
"Mysteries of the Ancient World" exhibit ...

Rubric:
1. Includes exactly five distinct artifacts.            [Hard Rule]
2. Each from a different culture and time period.       [Hard Rule]
3. Brief description of each artifact's significance.   [Hard Rule]
6. Communicates clearly and is well-organized.          [Principle]
8. Uses engaging language that stimulates curiosity.    [Principle]
```

`[Hard Rule]` = atomic, must-pass checklist items; `[Principle]` = softer quality criteria. The tags encode priority (numbers also work). Subcomponents of a rubric contribute to the score.

Per-prompt generation via a meta-prompt

Rubrics are generated per prompt to ensure quality and robustness to over-optimization. E.g. a per-domain base rubric, refined per-prompt by a supervising LM (Gunjal et al., 2025).

```
You are an expert rubric writer for science questions ...
Choose 7–20 rubric items based on question complexity.
Each item: title (2–4 words), description (category prefix +
  what to look for), weight.
  - Essential : critical facts; omission invalidates the answer  (1–5)
  - Important : key reasoning / completeness                     (1–5)
  - Optional  : nice-to-have depth or style                      (1–5)
  - Pitfall   : common mistakes to penalize                    (-1,-2)
Output: a JSON array of {title, description, weight}.
```

(Truncated – real meta-prompts are long and tuned to the training setup.)

Where rubrics are going

Rubric-based RL is a frontier of AI-feedback-driven training, expanding beyond its early uses:

- **Advanced instruction-following** (He et al., 2025)
- **Deep research** agents (Shao et al., 2025)
- **Evaluating** research agents (Sharma et al., 2025)
- **Long-form generation** with structured checklists (Ruan et al., 2025)

Very general tool!

Conclusions

Synthetic data is the single most common tool used by researchers (other than building infra) to make great models

The techniques surveyed here will continue to grow in complexity, and it's super fun to see. When I started writing this book, it was still a struggle to set up some synthetic data workflows! Knifecuts can happen, but overall it's a well-known workflow now.

There are many, minute open questions on how best to do this, but it often is a domain specific reflection of the technical problem at hand.

Course outline

1. Introduction & training overview – Chapters 1-3
2. IFT, reward models, rejection sampling – Chapters 4, 5, 9
3. RL theory – Chapter 6 (Part 1)
4. RL implementation & practice – Chapter 6 (Part 2)
5. Reasoning – Chapter 7
6. Direct alignment algorithms – Chapter 8
7. Synthetic data – Chapter 12
8. Preferences & preference data – Chapters 10/11

...

Thank you

Questions / discussion

Contact: nathan@natolambert.com

Newsletter: interconnects.ai

rlhfbook.com



References (1/6)

Agarwal, R., Vieillard, N., Zhou, Y., Stanczyk, P., Ramos Garea, S., et al. "*On-policy distillation of language models: Learning from self-generated mistakes*." *The Twelfth International Conference on Learning Representations*, 2024. [link]

Arora, K., El Asri, L., Bahuleyan, H., and Cheung, J. "*Why Exposure Bias Matters: An Imitation Learning Perspective of Error Accumulation in Language Generation*." *Findings of the Association for Computational Linguistics: ACL 2022*, 2022. [link]

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., et al. "*Constitutional ai: Harmlessness from ai feedback*." *arXiv preprint arXiv:2212.08073*, 2022.

Beeching, E., Huang, S., Jiang, A., Li, J., Lipkin, B., et al. "*NuminaMath 7B TIR*." *Hugging Face repository*, 2024.

Castricato, L., Lile, N., Anand, S., Schoelkopf, H., Verma, S., et al. "*Suppressing Pink Elephants with Direct Principle Feedback*." 2024.

Cui, G., Yuan, L., Ding, N., Yao, G., Zhu, W., et al. "*Ultrafeedback: Boosting language models with high-quality feedback*." 2023.

DeepSeek-AI. "*DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*." 2026. [link]

Feng, Y., Dohmatob, E., Yang, P., Charton, F., and Kempe, J. "*Beyond model collapse: Scaling up with synthesized data requires reinforcement*." *ICML 2024 Workshop on Theoretical Foundations of Foundation Models*, 2024.

References (2/6)

Gerstgrasser, M., Schaeffer, R., Dey, A., Rafailov, R., Sleight, H., et al. "*Is model collapse inevitable? breaking the curse of recursion by accumulating real and synthetic data*." *arXiv preprint arXiv:2404.01413*, 2024.

Gu, Y., Dong, L., Wei, F., and Huang, M. "*MiniLLM: Knowledge Distillation of Large Language Models*." *The Twelfth International Conference on Learning Representations*, 2024. [link]

Guha, E., Marten, R., Keh, S., and others. "*OpenThoughts: Data Recipes for Reasoning Models*." *arXiv preprint arXiv:2506.04178*, 2025.

Gunjal, A., Wang, A., Lau, E., Nath, V., He, Y., et al. "*Rubrics as Rewards: Reinforcement Learning Beyond Verifiable Domains*." 2025. [link]

He, Y., Li, W., Zhang, H., Li, S., Mandyam, K., et al. "*AdvancedIF: Rubric-Based Benchmarking and Reinforcement Learning for Advancing LLM Instruction Following*." 2025. [link]

Hinton, G., Vinyals, O., and Dean, J. "*Distilling the knowledge in a neural network*." *arXiv preprint arXiv:1503.02531*, 2015.

Jiao, X., Yin, Y., Shang, L., Jiang, X., Chen, X., et al. "*TinyBERT: Distilling BERT for Natural Language Understanding*." 2020. [link]

Kim, Y., and Rush, A. "*Sequence-Level Knowledge Distillation*." *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016. [link]

References (3/6)

Kim, S., Shin, J., Cho, Y., Jang, J., Longpre, S., et al. "*Prometheus: Inducing fine-grained evaluation capability in language models.*" *International Conference on Learning Representations*, 2024.

Kim, S., Suk, J., Longpre, S., Lin, B., Shin, J., et al. "*Prometheus 2: An open source language model specialized in evaluating other language models.*" *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., et al. "*Tülu 3: Pushing Frontiers in Open Language Model Post-Training.*" *arXiv preprint arXiv:2411.15124*, 2024.

Lee, H., Phatale, S., Mansoor, H., Lu, K., Mesnard, T., et al. "*Rlaif: Scaling reinforcement learning from human feedback with ai feedback.*" 2023.

Li, M., Zhang, Y., He, S., Li, Z., Zhao, H., et al. "*Superfiltering: Weak-to-strong data filtering for fast instruction-tuning.*" *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Liu, T., Xu, R., Yu, T., Hong, I., Yang, C., et al. "*OpenRubrics: Towards Scalable Synthetic Rubric Generation for Reward Modeling and LLM Alignment.*" 2025. [link]

LLM-Core Xiaomi. "*MiMo-V2-Flash Technical Report.*" 2025. [link]

Lu, K., and Thinking Machines Lab. "*On-Policy Distillation.*" *Thinking Machines Lab: Connectionism*, 2025. [link]

Mahan, D., Phung, D., Rafailov, R., Blagden, C., Lile, N., et al. "*Generative Reward Models.*" 2024.

References (4/6)

Miranda, L., Wang, Y., Elazar, Y., Kumar, S., Pyatkin, V., et al. "*Hybrid Preferences: Learning to Route Instances for Human vs. AI Feedback*." *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025. [link]

Panickssery, A., Bowman, S., and Feng, S. "*Llm evaluators recognize and favor their own generations*." *Advances in Neural Information Processing Systems*, 2024.

Penaloza, E., Vattikonda, D., Gontier, N., Lacoste, A., Charlin, L., et al. "*Privileged Information Distillation for Language Models*." 2026. [link]

Rezaei, M., Vacareanu, R., Wang, Z., Wang, C., Liu, B., et al. "*Online Rubrics Elicitation from Pairwise Comparisons*." 2025. [link]

Ross, S., Gordon, G., and Bagnell, D. "*A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning*." *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011. [link]

Ruan, J., Nair, I., Cao, S., Liu, A., Munir, S., et al. "*ExpertLongBench: Benchmarking Language Models on Expert-Level Long-Form Generation Tasks with Structured Checklists*." 2025. [link]

Sanh, V., Debut, L., Chaumond, J., and Wolf, T. "*DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter*." 2019. [link]

Schulman, J. "*Approximating KL Divergence*." 2020.

References (5/6)

Shao, R., Asai, A., Shen, S., Ivison, H., Kishore, V., et al. "*DR Tulu: Reinforcement Learning with Evolving Rubrics for Deep Research*." 2025. [link]

Sharma, A., Keh, S., Mitchell, E., Finn, C., Arora, K., et al. "*A Critical Evaluation of AI Feedback for Aligning Large Language Models*." *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Sharma, M., Zhang, C., Bandi, C., Wang, C., Aich, A., et al. "*ResearchRubrics: A Benchmark of Prompts and Rubrics For Evaluating Deep Research Agents*." 2025. [link]

Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., et al. "*AI models collapse when trained on recursively generated data*." *Nature*, 2024.

Song, M., and Zheng, M. "*A Survey of On-Policy Distillation for Large Language Models*." 2026. [link]

Sun, Z., Shen, Y., Zhang, H., Zhou, Q., Chen, Z., et al. "*SALMON: Self-Alignment with Principle-Following Reward Models*." *The Twelfth International Conference on Learning Representations*, 2024. [link]

Team, C. "*Introducing Composer 2.5*." *Cursor Blog*, 2026. [link]

Viswanathan, V., Sun, Y., Ma, S., Kong, X., Cao, M., et al. "*Checklists Are Better Than Reward Models For Aligning Language Models*." 2025. [link]

References (6/6)

Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N., et al. "*Self-instruct: Aligning language models with self-generated instructions.*" *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

Wang, P., Li, L., Chen, L., Cai, Z., Zhu, D., et al. "*Large language models are not fair evaluators.*" *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Xu, Y., Chakraborty, T., Kıcıman, E., Aryal, B., Rodrigues, E., et al. "*RLTHF: Targeted Human Feedback for LLM Alignment.*" *International Conference on Machine Learning (ICML)*, 2025. [link]

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., et al. "*Qwen3 technical report.*" *arXiv preprint arXiv:2505.09388*, 2025.

Zhao, S., Xie, Z., Liu, M., Huang, J., Pang, G., et al. "*Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models.*" 2026. [link]